Chapter 1 Java Language Overview

Ch1. Language Basics (2)

杨巍

yangwei@mail.neu.edu.cn



Variables

- declaring a local variable
  - All variables must first be declared before they can be used

```
Data-types  variable-name;
```

```
int   num;
```

```
int   num,num1,num2;
```

Naming

- Always begin your variable names with a letter, not "$" or "_“
- Subsequent characters may be letters, digits, dollar signs, or underscore characters.
- Variable names are case-sensitive.
- Avoid using keywords

Key words

abstract	default	if	protected	throws
assert	do	implements	public	transient
boolean	double	import	return	try
break	else	instanceof	short	void
byte	extends	int	static	volatile
case	enum	interface	synchronized	while
catch	final	long	super	
char	fianlly	native	strictfp	
class	float	new	switch	
continue	for	package	this	
const	goto	private	throw	

Naming

- If the name you choose consists of only one word, spell that word in all lowercase letters.

```
int year;
```

- If it consists of more than one word, capitalize the first letter of each subsequent word.

```
int currentGear;
```

Initializing declaration

- Initializing declaration
  - Declaration that also assigns an initial value

```
Data-types  variable-name;
variable-name=initial-value;
```

```
int   num;
num=9;
```

```
int   num=9;
```

Data type

- A variable's data type determines the values it may contain, plus the operations that may be performed on it.
- Data type
  - primitive types
    - Integral number
    - floating point number
    - char
    - boolean
  - reference types

Primitive types

- Integral number

Data type	Size	Min-max
byte	8 bits	$-2^7 \sim 2^7-1$
short	16 bits	$-2^{15} \sim 2^{15}-1$
int	32 bits	$-2^{31} \sim 2^{31}-1$
long	64 bits	$-2^{63} \sim 2^{63}-1$

Integer Literals

- **Integer Literals**
  - An integer literal is of type long if it ends with the letter L or l;
  - otherwise it is of type int.
  - Values of the integral types byte, short, int, and long can be created from int literals.
  - Values of type long that exceed the range of int can be created from long literals.

Integer Literals

- Integer literals can be expressed by these number systems
  - Decimal
    - Base 10
  - Hexadecimal
    - Base 16
  - Binary
    - Base 2

Primitive types

- Floating point number

Data type	Size	Min-max
float	32 bits	-3.403E38 ~ 3.403E38
double	64 bits	-1.798E308 ~ 1.798E308

Floating-Point Literals

- Floating-Point Literals
  - A floating-point literal is of type **float** if it ends with the letter F or f;
  - otherwise its type is **double** and it can optionally end with the letter D or d.

Floating-Point Literals

- Floating-Point Literals
  - For scientific notation
    - The floating point types (float and double) can also be expressed using E or e

```
public class DataDemo{
        public static void main(String[ ] args){
                double d1 = 123.4;
               double d2=1.234e2;
        }
}
```

boolean

- The boolean data type has only two possible values
  - true and false
  - Use this data type for simple flags that track true/false conditions.

char

- char
  - The char data type is a single 16-bit Unicode character.
  - Java采用Unicode字符码系统
  - Unicode为每个字符制订了一个唯一的数值
  - Always use 'single quotes' for char literals

char

- Unicode escape sequences

escape sequences	meaning
\r	carriage return
\n	line feed
\t	tab
\b	backspace
\’	single quote
\”	double quote
\\	backslash

Summary of Variables

- The eight primitive data types are
  - byte, short, int, long
  - float, double
  - boolean, and char.
- A literal is the source code representation of a fixed value.

Operators

- Java was inherited from C++, most of these operators will be familiar to C and C++ programmers.
- Java has also added some improvements and simplifications.
- An operator takes one or more arguments and produces a new value.

Operators

- **Assignment Operator**
  - =
- **The Arithmetic Operators**

Operator	Description
+	Additive operator (also used for String concatenation)
-	Subtraction operator
*	Multiplication operator
/	Division operator
%	Remainder operator

Operators

- **The Equality and Relational Operators**

Operator	Description
==	equal to
!=	not equal to
>	greater than
>=	greater than or equal to
<	less than
<=	less than or equal to

Operators

- **The Conditional Operators**

Operator	Description
&&	Conditional-AND
||	Conditional-OR

  - These operators exhibit "short-circuiting" behavior, which means that the second operand is evaluated only if needed

Operators

- **Bitwise and Bit Shift Operators**
  - perform bitwise and bit shift operations on integral types
  - The operators discussed in this section are less commonly used.
  - The intent is to simply make you aware that these operators exist.

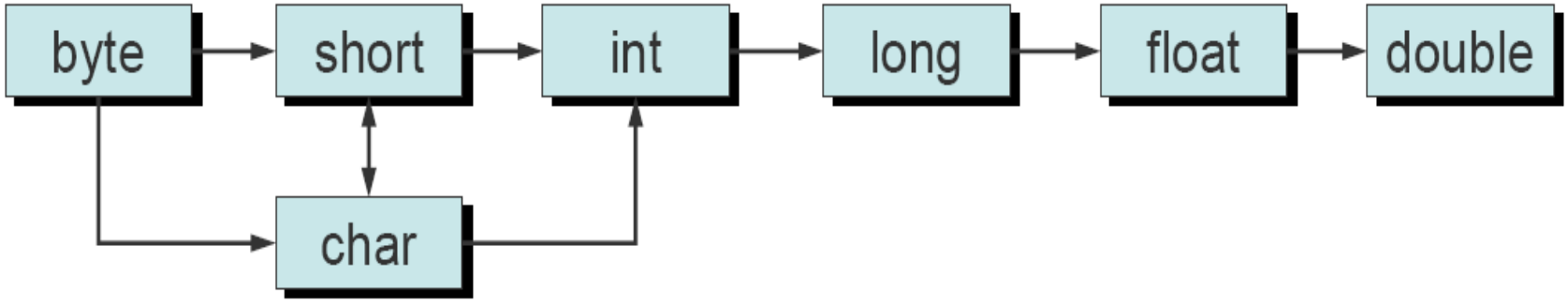
Operators

- **Bitwise and Bit Shift Operators**

Operator	Description
&	bitwise AND operation
^	bitwise exclusive OR operation
|	bitwise inclusive OR operation

Operators

- Type conversion
  - Numbers are automatically promoted to a more inclusive type if no information is lost.



Operators

```
// AutoChange.java
public class AutoChange{
        public static void main(String[ ] args){
                int x=10;
                float y=3;
                System.out.println("x/y="+x/y);
        }
}
```

Operators

- Casting operator
  - Casting allows you to make this type conversion explicit, or to force it when it wouldn't normally happen.
  - Cast should be used sparingly and with care.

(converted-type)variable;

Operators

```
//Casting.java
public class Casting{
        public static void main(String[ ] args){
                int x=10;
                int y=3;
                System.out.println("x/y="+ (float)x/y );
        }
}
```

Operators

- Casting operator
  - Java allows you to cast any primitive type to any other primitive type, except for boolean, which doesn't allow any casting at all.
  - Class types do not allow casting.

Operators

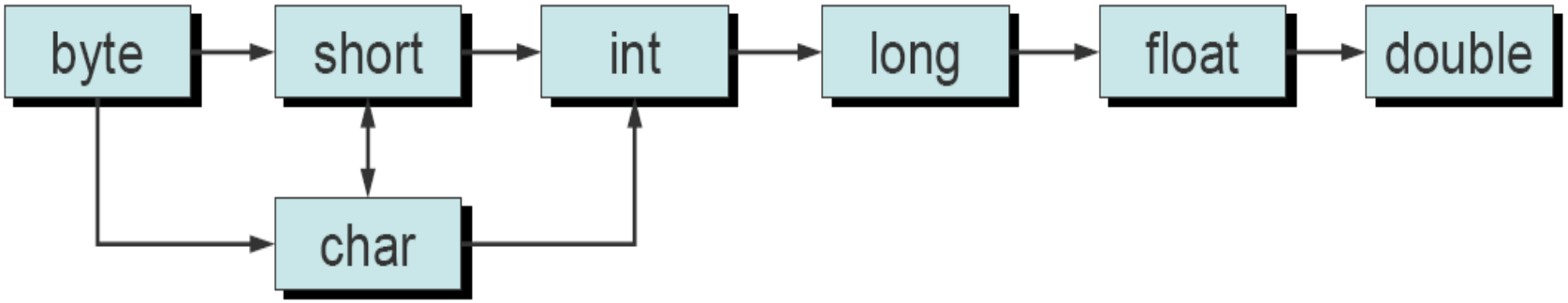
- Precedence
  - defines how an expression evaluates when several operators are present.
  - Java has specific rules that determine the order of evaluation.
  - multiplication and division happen before addition and subtraction
  - use parentheses to make the order of evaluation explicit

Summary

Data Type	Bits	Valid value
char	16	\u0000~\uffff
byte	8	-128~127
short	16	-32768~32767
int	32	-2147483648~2147483647
long	64	-9223372036854775808~9223372036854775807
float	32	1.40239846E-45~3.40282347E+38
double	64	4.94065645841246544E-324 ~1.79769313486231570E+308
boolean	1	false ~ true

Summary

- Promotion
  - Automatic



  - Casting operator

if Statements

Note that Java doesn't allow you to use a number as a boolean.

```
if (Boolean-expression){
        statement;
}
```

```
if (Boolean-expression){
        statement1;
}else{
        statement2;
}
```

```
if (Boolean-expression) {
        statements1;
} else if (Boolean-expression) {
        statements2;
} else if (Boolean-expression) {
        statements3;
} else{
        statements4;
}
```

if Statements

- Ternary if-else operator

```
boolean-exp ?expression1:exprssion2
```

```
if (Boolean-expression){
        statement1;
}else{
        statement2;
}
```

switch Statements

- switch Statements
  - selection statement
  - have a number of possible execution paths.
  - Switch语句中选择值只能是字符或整形常量
  - each case ends with a break
  - But the break is optional

switch Statements

```
switch (integral-selector){
    case integral-value1:
            statements;
            break;

    case integral-value2 :
            statements;
            break;
    ……
    case integral-valueN :
            statements;
            break;
    default:
            statements;
}
```

switch Statements

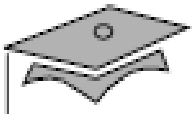
- **Using Strings in switch Statements**
  - In Java SE 7 and later, you can use a String object in the switch statement's expression.

```
//SwitchDemo.java
public class SwitchDemo{
        public static void main(String[ ] args){
                String day="Monday";
                switch(day){
                        case "Monday" :{
                                System.out.println(1);break;}
                        case "Tuesday":{
                                System.out.println(2);break;}
                        case "Wednesday":{
                                System.out.println(3);break;}
                        case "Thursday" :{
                                System.out.println(4);break;}
                        case "Friday" :{
                                System.out.println(5);break;}
                        default:{ System.out.println("rest"); }
                }
        }
}
```

while Statements

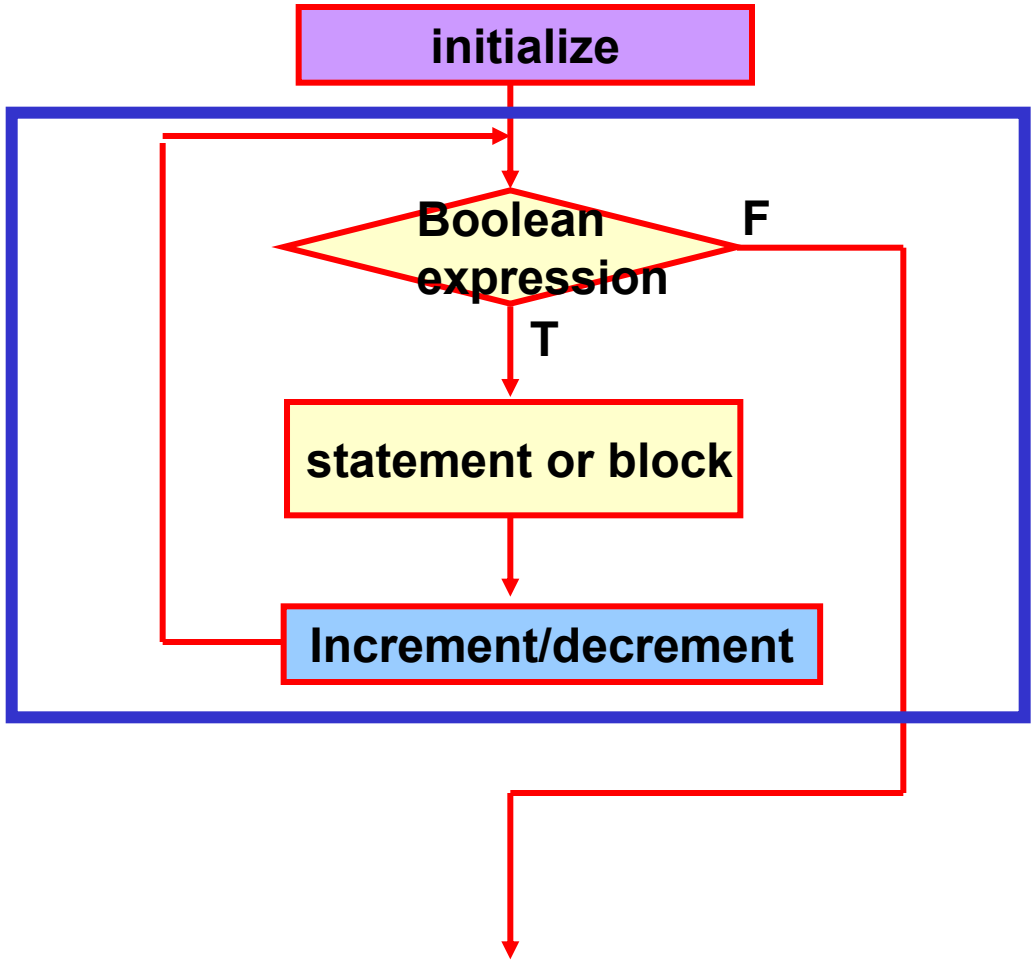
- 事先不知道循环该执行多少次
- While语句先判断，后执行

```
while (Boolean-expression){
        statements;
        increment/decrement counter;
}
```



Obj

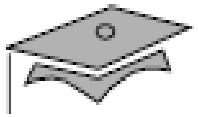
```
while (Boolean-expression){
        statements;
        increment/decrement counter;
}
```



do…while Statements

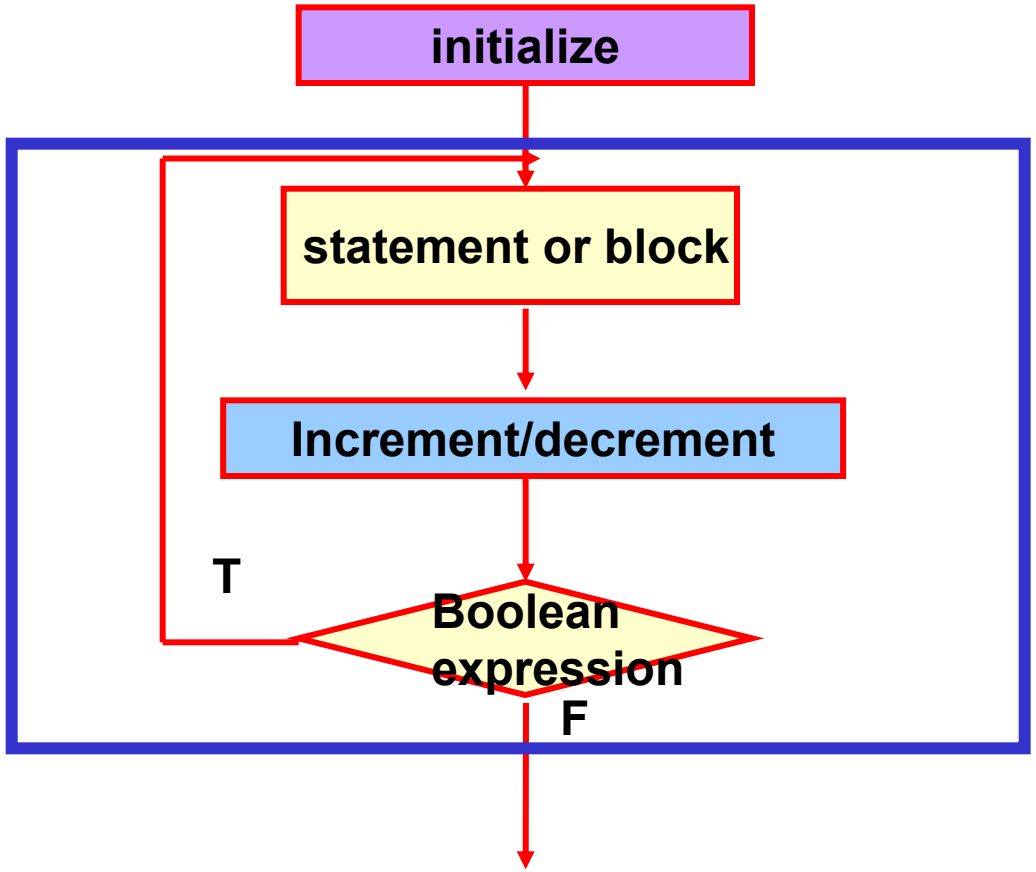
- do…while
  - The sole difference between while and do-while is that the statement of the do-while always executes at least once, even if the expression evaluates to false the first time

```
do {
        statements;
        increment/decrement counter;
} while (Boolean-expression);
```



Ob

```
do {
        statements;
        increment/decrement counter;
} while (Boolean-expression);
```



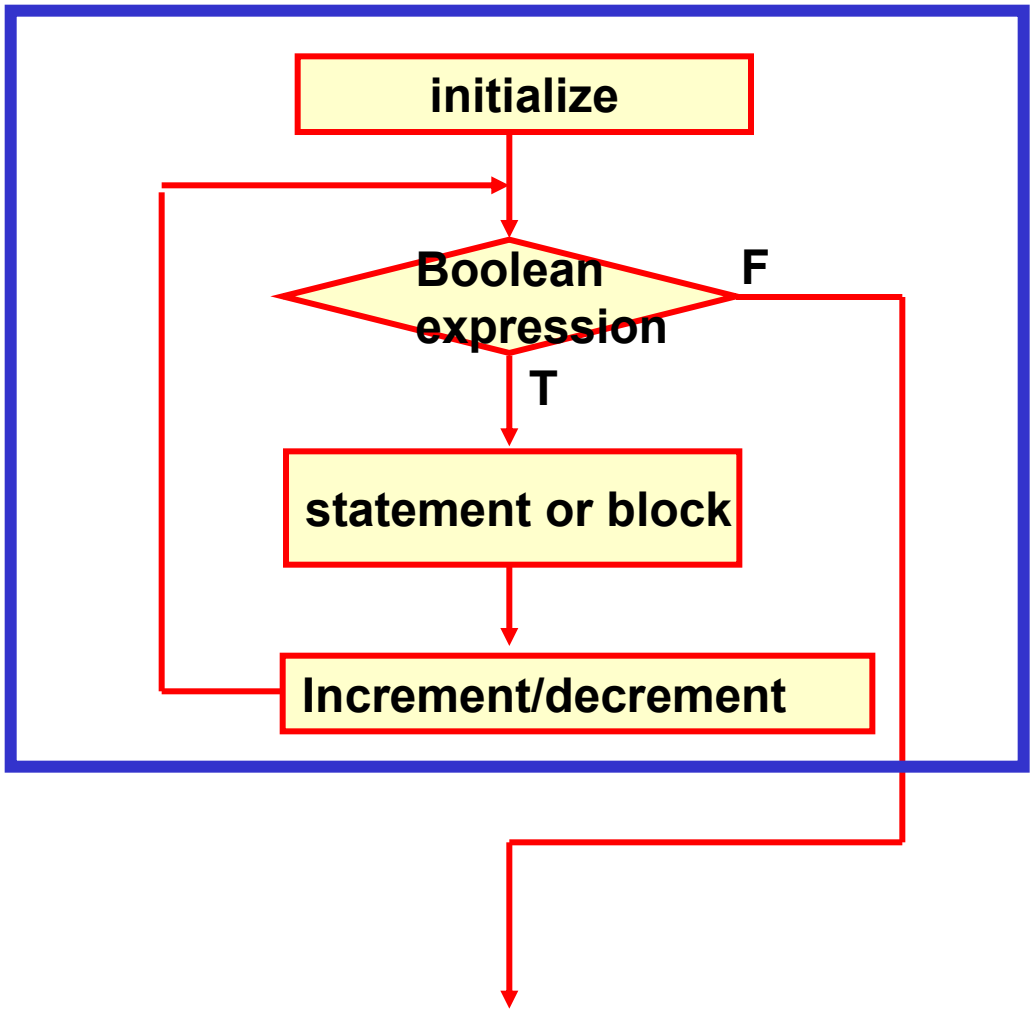
for Statements

```
for(Expression1;Expression2;Expression3) {
        statements;
}
```

```
for (initialize; Boolean-expression; increment/decrement) {
        statements;
}
```

```
Expression1;
while( Expression2 ) {
        Block;
        Expression3
};
```

for Statements



Break and continue

- Break statement
  - Immediately exits the loops
- Continue statement
  - Immediately begins the next iteration of the loop

Scoping

- Scoping
  - Determines both the visibility and lifetime of the names defined within that scope.
  - In C, C++, and Java, scope is determined by the placement of curly braces {}
  - Determination comes entirely from the location in which the variable is declared — which is between the opening and closing braces of a method.

Scoping

```
{
        int x = 12;
        {
                int x = 96; // Illegal
        }
}
```

The compiler will announce that the variable x has already been defined.

Comments

- Comments
  - Comments are ignored by the compiler but are useful to other programmers.
  - The Java programming language supports three kinds of comments
    - Comments on one line
    - Comments on one or more lines
    - Documentation comments

Comments

- Comments on one line
  - The compiler ignores everything from // to the end of the line.

```
// This is comment on one line
int num = 10;   // declare a variable named num
```

Comments

- Comments on one or more lines
  - The compiler ignores everything from /* to */.

```
/*
        int  = 10; //
        int x=5;
*/
```

Comments

- Documentation comments
  - 以/**开头，以*/结尾，中间可以加入特殊的指令
  - 文档注释放在类、变量和方法之前，说明其作用

```
/**
 * Title: test
 * @author yw
 * @version 1.0
 */
```

Comments

- Documentation comments(*doc comment)*
  - The compiler ignores this kind of comment, just like it ignores comments that use /* and */.
  - The javadoc tool uses doc comments when preparing automatically generated documentation.

javadoc 类文件(带扩展名)

The aims of this session

- Java basic
  - Variable
  - Datatype
  - Operators
  - Control structure
  - Comments